API 사용법

개요

AMD SEV 라이브러리는 SEV 플랫폼 관리와 게스트 VM 런치를 위한 포괄적인 API를 제공합니다. 이 문서는 주요 API 함수들의 사용법, 호출 순서, 그리고 베스트 프랙티스를 설명합니다.

기본 설정

의존성 추가

`Cargo.toml` 에 다음 의존성을 추가합니다:

```
[dependencies]

sev = { version = "1.2.1", features = ["openssl"] }
```

기능 플래그

- `sev` : 레거시 SEV 및 SEV-ES 지원
- `snp` : SEV-SNP 지원
- `openssl` : 암호화 검증 기능 (권장)
- `crypto_nossl` : OpenSSL 없는 순수 Rust 암호화

플랫폼 관리 API

1. 펌웨어 핸들 생성

모든 플랫폼 관리 작업은 `Firmware` 핸들을 통해 수행됩니다.

```rust
use sev::firmware::host::Firmware;

// SEV 디바이스 열기 (/dev/sev)
let mut firmware = Firmware::open()?;
```

매개변수: 없음

반환값: `Result<Firmware, std::io::Error>`

권한: `/dev/sev` 디바이스에 대한 읽기/쓰기 권한 필요

2. 플랫폼 상태 조회

```rust
use sev::firmware::host::{Firmware, Status};

let mut firmware = Firmware::open()?;
let status: Status = firmware.platform_status()?;

println!("플랫폼 상태: {:?}", status.state);
println!("게스트 수: {}", status.guests);
println!("펌웨어 버전: {}.{}",
status.build.version.major,
status.build.version.minor);
```

매개변수: `&mut self`

반환값: `Result<Status, UserApiError>`

용도: 플랫폼 초기화 상태, 활성 게스트 수, 펌웨어 버전 확인

3. 플랫폼 초기화

```
// 플랫폼 리셋 (모든 게스트 종료)
firmware.platform_reset()?;

// PEK (Platform Encryption Key) 생성
firmware.pek_generate()?;

// PDH (Platform Diffie-Hellman) 키 쌍 생성
firmware.pdh_generate()?;
```

호출 순서: `platform_reset()` → `pek_generate()` → `pdh_generate()`

4. 인증서 관리

```
use sev::certs::sev::sev::{Certificate, Chain};

// 인증서 체인 내보내기
let  chain: Chain = firmware.pdh_cert_export()?;

// PEK CSR (Certificate Signing Request) 생성
let  pek_cert: Certificate = firmware.pek_csr()?;
```

```
// 인증서 가져오기 (플랫폼 소유권 획득)
firmware.pek_cert_import(&pek_cert, &oca_cert)?;
```

인증서 체인 구조: ARK → ASK → CEK → PEK → PDH

5. 플랫폼 식별자 조회

```
use sev::firmware::host::Identifier;

let identifier: Identifier = firmware.get_identifier()?;
println!("플랫폼 ID: {}", identifier);
```

용도: AMD KDS에서 서명된 CEK 인증서 요청 시 사용

SEV-SNP 플랫폼 관리

1. SNP 플랫폼 상태 조회

```
use sev::firmware::host::SnpPlatformStatus;

let snp_status: SnpPlatformStatus = firmware.snp_platform_status()?;
println!("SNP 상태: {:?}", snp_status);
```

2. SNP 설정 관리

```rust
use sev::firmware::host::{Config, TcbVersion};

// 새로운 설정 생성
let  config = Config::new(
TcbVersion::new(3, 0, 10, 169), // TCB 버전
0, // 추가 플래그
);

// 설정 적용
firmware.snp_set_config(config)?;

// 설정 커밋
firmware.snp_commit()?;
```

3. VLEK Hashstick 로드

```rust
use sev::{Generation, firmware::host::WrappedVlekHashstick};

// 호스트 세대 식별
let  generation = Generation::identify_host_generation()?;

// Hashstick 파싱
let  hashstick = WrappedVlekHashstick::from_bytes(
```

```
hashstick_bytes.as_slice(),
generation
)?;


// Hashstick 로드
firmware.snp_vlek_load(hashstick)?;
```

게스트 런치 API

SEV 게스트 런치

1. 런처 초기화

```
use sev::launch::sev::{Launcher, New};
use std::fs::OpenOptions;


// KVM 및 SEV 파일 디스크립터 열기
let  kvm_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/kvm")?;


let  sev_fd = OpenOptions::new()
.read(true)
```

```
.write(true)
.open("/dev/sev")?;

// SEV 런처 생성
let launcher: Launcher<New, _, _> = Launcher::new(kvm_fd, sev_fd)?;

// SEV-ES 런처 생성 (선택사항)
let launcher_es: Launcher<New, _, _> = Launcher::new_es(kvm_fd, sev_fd)?;
```

2. 게스트 시작

```
use sev::launch::sev::{Start, Policy, PolicyFlags, Session};
use sev::firmware::host::Version;

// 정책 설정
let policy = Policy {
flags: PolicyFlags::NO_DEBUG | PolicyFlags::ENCRYPTED_STATE,
minfw: Version { major: 1, minor: 0 },
};

// 세션 정보 (실제 구현에서는 적절한 값 설정 필요)
let session = Session {
nonce: [0u8; 16],
```

```
wrap_tk: [0u8; 32],
wrap_iv: [0u8; 16],
wrap_mac: [0u8; 32],
policy_mac: [0u8; 32],
};

// 시작 매개변수
let  start = Start {
policy,
cert: tenant_cert, // 테넌트 DH 인증서
session,
};

// 게스트 시작
let  launcher = launcher.start(start)?;
```

3. 데이터 업데이트

```
// 게스트 메모리 암호화
launcher.update_data(&guest_memory_data)?;

// SEV-ES의 경우 VMSA 업데이트
launcher.update_vmsa()?;
```

4. 측정 및 완료

```rust
use sev::launch::sev::Measurement;

// 측정 요청
let launcher = launcher.measure()?;
let measurement: Measurement = launcher.measurement();

// 시크릿 주입 (선택사항)
launcher.inject(&secret, guest_address)?;

// 런치 완료
let handle = launcher.finish()?;
```

SEV-SNP 게스트 런치

1. SNP 런처 초기화

```rust
use sev::launch::snp::{Launcher, New, Start};
use sev::firmware::guest::GuestPolicy;

// SNP 런처 생성
let launcher: Launcher<New, _, _> = Launcher::new(kvm_fd, sev_fd)?;
```

```rust
// 게스트 정책 설정
let guest_policy = GuestPolicy::default();


// 시작 매개변수
let start = Start::new(guest_policy, [0u8; 16]); // GOSVW


// 게스트 시작
let launcher = launcher.start(start)?;
```

2. SNP 데이터 업데이트

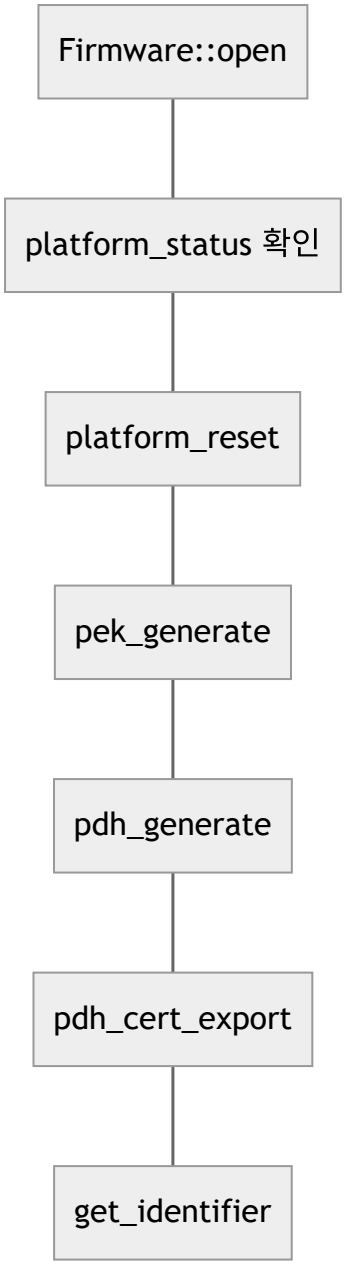
```rust
use sev::launch::snp::{Update, PageType};


// 페이지 업데이트
let update = Update::new(
start_gfn, // 게스트 프레임 번호
&memory_data, // 메모리 데이터
PageType::Normal, // 페이지 타입
);


launcher.update_data(update, gpa, gpa_len)?;
```

3. SNP 런치 완료

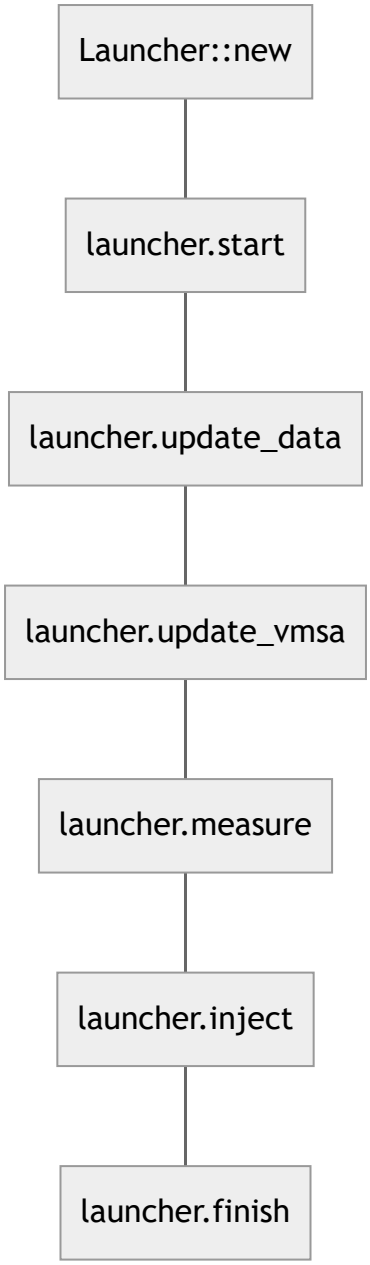
```rust
use sev::launch::snp::Finish;


// 완료 매개변수
let  finish = Finish::new(
Some(&id_block), // ID 블록 (선택사항)
Some(&id_auth), // ID 인증 (선택사항)
host_data, // 호스트 데이터
);


// 런치 완료
let (kvm_fd, sev_fd) = launcher.finish(finish)?;
```

API 호출 순서 및 의존성

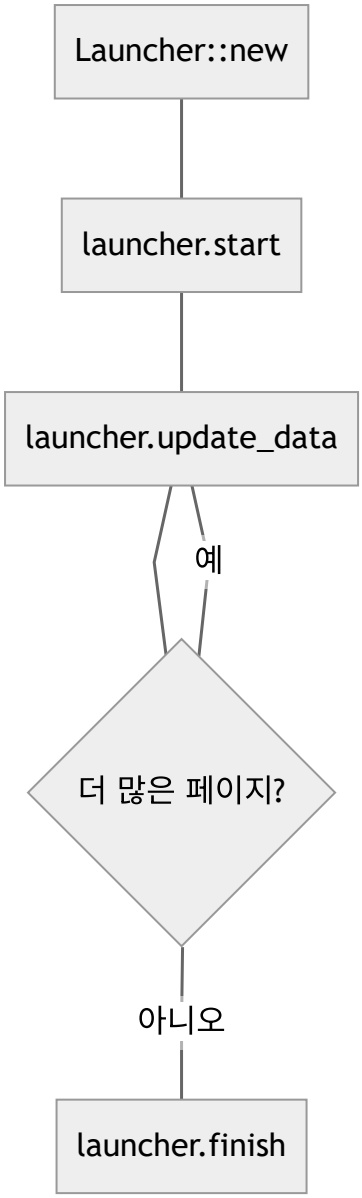
플랫폼 초기화 순서


Firmware::open
platform_status 확인
platform_reset
pek_generate
pdh_generate
pdh_cert_export
get_identifier

SEV 게스트 런치 순서



SEV-SNP 게스트 런치 순서



일반적인 사용 패턴

1. 플랫폼 상태 확인 패턴

```
fn check_platform_ready(firmware: &mut Firmware) -> Result<bool, UserApiError>
let status = firmware.platform_status()?;
match status.state {
State::Uninitialized => {
println!("플랫폼이 초기화되지 않음");
```

```
Ok(false)
},
State::Initialized => {
println!("플랫폼 초기화됨, 게스트 런치 가능");
Ok(true)
},
State::Working => {
println!("플랫폼 작업 중, {} 게스트 활성", status.guests);
Ok(true)
},
}
}
```

2. 오류 처리 패턴

```
use sev::error::{UserApiError, FirmwareError, SevError};

fn handle_sev_error(error: UserApiError) {
match error {
UserApiError::FirmwareError(FirmwareError::KnownSevError(sev_err)) => {
match sev_err {
SevError::InvalidPlatformState => {
println!("플랫폼 상태가 올바르지 않습니다. 플랫폼을 초기화하세요.");
```

```
},
SevError::AlreadyOwned => {
println!("플랫폼이 이미 소유되어 있습니다.");
},
_ => println!("SEV 오류: {}", sev_err),
}
},
UserApiError::IOError(io_err) => {
println!("I/O 오류: {}. 권한을 확인하세요.", io_err);
},
_ => println!("기타 오류: {}", error),
}
}
```

3. 리소스 관리 패턴

```
use std::fs::File;

struct SevManager {
firmware: Firmware,
kvm_fd: File,
}
```

```
impl SevManager {
fn new() -> Result<Self, Box<dyn std::error::Error>> {
let firmware = Firmware::open()?;
let kvm_fd = OpenOptions::new()
.read(true)
.write(true)
.open("/dev/kvm")?;
Ok(Self { firmware, kvm_fd })
}
fn initialize_platform(&mut self) -> Result<(), UserApiError> {
self.firmware.platform_reset()?;
self.firmware.pek_generate()?;
self.firmware.pdh_generate()?;
Ok(())
}
}
```

베스트 프랙티스

1. 오류 처리

- 모든 API 호출에서 `Result` 타입을 적절히 처리
- `UserApiError` 의 세부 타입에 따른 구체적인 오류 처리

- 복구 가능한 오류에 대한 재시도 로직 구현

2. 리소스 관리

- 파일 디스크립터의 적절한 생명주기 관리
- RAII 패턴을 활용한 자동 리소스 해제
- 메모리 버퍼의 안전한 할당 및 해제

3. 보안 고려사항

- 민감한 데이터(키, 시크릿)의 안전한 처리
- 메모리 내 민감 정보의 적절한 소거
- 권한 최소화 원칙 적용

4. 성능 최적화

- 불필요한 API 호출 최소화
- 배치 처리를 통한 효율성 향상
- 메모리 복사 최소화

5. 디버깅 및 로깅

- 적절한 로그 레벨 설정
- 오류 상황에 대한 상세한 컨텍스트 정보 제공
- 디버그 빌드에서의 추가 검증 로직

다음 단계

이 API 사용법 문서를 숙지한 후, 다음 문서들을 참조하여 구체적인 구현을 진행하세요:

- 코드 예제: 실제 동작하는 완전한 예제 코드
- 오류 처리: 상세한 오류 처리 가이드
- API 레퍼런스: 모든 API의 상세 레퍼런스